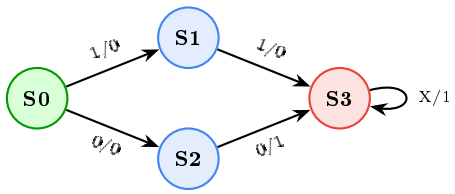
Digital System Design

Assignment 1 - Complete Solutions



Course: Digital System Design (Theory)

Reference: Digital Design, First Edition
by Frank Vahid (Wiley, 2007)
Chapters 2, 3, and 4

Topics: Combinational Logic Design
Finite State Machines
Sequential Circuit Design

Prepared: March 24, 2026

Contents

I Theoretical Foundation **2**

1 Chapter 2: Combinational Logic Design **2**
1.1 The Combinational Design Process . . . 2
1.2 Decoders . . . 2
1.2.1 2-to-4 Decoder with Enable . . . 2
1.2.2 Truth Table for 2×4 Decoder . . . 3
1.3 Multiplexers . . . 3

2 Chapter 3: Sequential Logic Design - Controllers **4**
2.1 Finite State Machines (FSMs) . . . 4
2.2 FSM Types . . . 4
2.3 FSM Design Process . . . 4
2.4 State Diagram Notation . . . 5

3 Chapter 4: Datapath Components **6**
3.1 Registers and Counters . . . 6
3.2 Controller-Datapath Architecture . . . 6

II Assignment Solutions **7**

4 Question 1: Fuel Sensor System (20 marks) **7**
4.1 Understanding the Problem . . . 7
4.2 Truth Table . . . 7
4.3 Design Approach Using 2×4 Decoders . . . 7
4.4 Boolean Equations . . . 8
4.5 Circuit Block Diagram . . . 8
4.6 Detailed Decoder Connections . . . 8

5 Question 2: Photo Booth FSM (10 marks) **9**
5.1 Input/Output Definition . . . 9
5.2 State Encoding . . . 9
5.3 FSM State Diagram . . . 10
5.4 State Transition Table . . . 10

6 Question 3: Sequence Generator FSM (10 marks) **11**
6.1 Part (a): Stop and Restart . . . 11
6.2 Part (b): 3-Cycle Postpone . . . 11
6.3 Part (c): 30-Cycle Delay with MSI Counter . . . 12

III Quiz Questions **13**

7 Multiple Choice **13**

8 Short Answer **13**

A Quick Reference **14**
A.1 Decoder Expansion 14
A.2 FSM Design Checklist 14

Part I
Theoretical Foundation

1 Chapter 2: Combinational Logic Design

Key Concepts - Combinational Circuits

Combinational circuits are digital circuits where outputs depend **only** on current inputs (no memory). The design process follows a structured approach from problem specification to gate implementation.

1.1 The Combinational Design Process

Step 1: Capture the Function - Create a truth table or Boolean equations

Step 2: Convert to Equations - Derive Sum-of-Products (SOP) or Product-of-Sums (POS)

Step 3: Minimize - Use K-maps or algebraic methods

Step 4: Implement - Draw the circuit using gates

1.2 Decoders

Decoder Definition

A **decoder** is a combinational circuit that converts an n-bit binary input into 2^n output lines, where exactly one output is active at a time.

1.2.1 2-to-4 Decoder with Enable

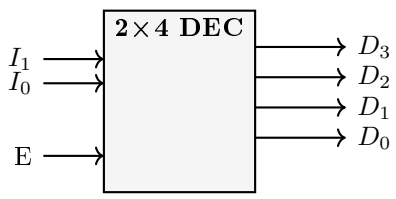


Figure 1: 2×4 Decoder with Enable Input

1.2.2 Truth Table for 2×4 Decoder

E	I_1	I_0	D_0	D_1	D_2	D_3
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

Table 1: 2×4 Decoder Truth Table

1.3 Multiplexers

A **multiplexer (MUX)** selects one of 2^n data inputs based on n select lines.

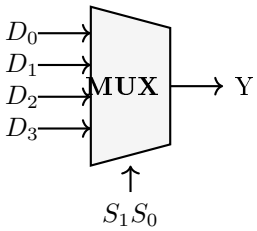


Figure 2: 4-to-1 Multiplexer Symbol

2 Chapter 3: Sequential Logic Design - Controllers

Key Concepts - Sequential Circuits

Sequential circuits have **memory** - outputs depend on both current inputs AND past inputs (stored state). The fundamental building block is the flip-flop.

2.1 Finite State Machines (FSMs)

An FSM is defined by the 5-tuple: $(Q, \Sigma, \delta, q_0, F)$

- Q = Finite set of states
- Σ = Input alphabet
- δ = Transition function: $Q \times \Sigma \to Q$
- q_0 = Initial state
- F = Set of final/accepting states

2.2 FSM Types

Type	Output Depends On	Output Timing
Moore Machine	Current state only	Output with state
Mealy Machine	Current state + input	Output with transition

Table 2: Comparison of Moore and Mealy FSMs

2.3 FSM Design Process

Step 1: Understand the Problem - Identify inputs, outputs, and behavior

Step 2: Draw State Diagram - Create states and transitions

Step 3: Create State Table - List all state transitions

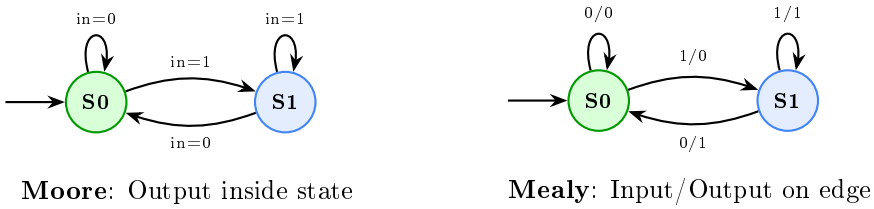
Step 4: Assign State Encoding - Binary codes to states

Step 5: Derive Next-State Logic - Boolean equations for flip-flops

Step 6: Derive Output Logic - Boolean equations for outputs

Step 7: Implement - Draw circuit with flip-flops and gates

2.4 State Diagram Notation



Moore: Output inside state

Mealy: Input/Output on edge

Figure 3: Moore vs Mealy Machine Notation

3 Chapter 4: Datapath Components

Key Concepts - Datapath

Datapaths process multi-bit data using registers, ALUs, and interconnections. Controllers (FSMs) direct datapaths to perform operations.

3.1 Registers and Counters

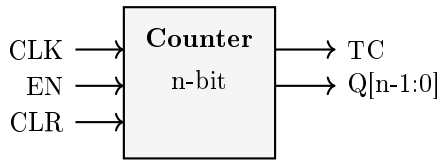


Figure 4: n-bit Counter with Terminal Count Output

3.2 Controller-Datapath Architecture

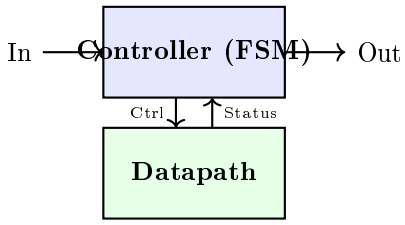


Figure 5: Controller-Datapath Architecture

Part II
Assignment Solutions

4 Question 1: Fuel Sensor System (20 marks)

Problem Statement

Design a combinational circuit for a fuel sensor system that detects fuel level using five sensors (S_0 to S_4). The output is a 3-bit binary representation of the fuel level. Use **five 2×4 decoders with enable inputs** and any additional logic gates.

4.1 Understanding the Problem

The fuel tank has 5 sensors positioned at different heights:

- S_0 = Bottom sensor (covered first)
- S_4 = Top sensor (covered last)
- Sensor = 1 when covered by fuel
- Valid patterns: Contiguous 1s from bottom

4.2 Truth Table

S_4	S_3	S_2	S_1	S_0	F_2	F_1	F_0	**Level**
0	0	0	0	0	0	0	0	Empty (0)
0	0	0	0	1	0	0	1	Level 1
0	0	0	1	1	0	1	0	Level 2
0	0	1	1	1	0	1	1	Level 3
0	1	1	1	1	1	0	0	Level 4
1	1	1	1	1	1	0	1	Full (5)

Table 3: Fuel Level Truth Table (Valid Patterns Only)

4.3 Design Approach Using 2×4 Decoders

Design Strategy

We use five 2×4 decoders to detect specific fuel levels. Each decoder's enable is controlled by higher-level sensor conditions.

4.4 Boolean Equations

Level detection signals:

$$L_0 = \overline{S_4} \cdot \overline{S_3} \cdot \overline{S_2} \cdot \overline{S_1} \cdot \overline{S_0} \tag{1}$$
$$L_1 = \overline{S_4} \cdot \overline{S_3} \cdot \overline{S_2} \cdot \overline{S_1} \cdot S_0 \tag{2}$$
$$L_2 = \overline{S_4} \cdot \overline{S_3} \cdot \overline{S_2} \cdot S_1 \cdot S_0 \tag{3}$$
$$L_3 = \overline{S_4} \cdot \overline{S_3} \cdot S_2 \cdot S_1 \cdot S_0 \tag{4}$$
$$L_4 = \overline{S_4} \cdot S_3 \cdot S_2 \cdot S_1 \cdot S_0 \tag{5}$$
$$L_5 = S_4 \cdot S_3 \cdot S_2 \cdot S_1 \cdot S_0 \tag{6}$$

Final outputs (binary encoding):

$$F_2 = L_4 + L_5 \tag{7}$$
$$F_1 = L_2 + L_3 \tag{8}$$
$$F_0 = L_1 + L_3 + L_5 \tag{9}$$

4.5 Circuit Block Diagram

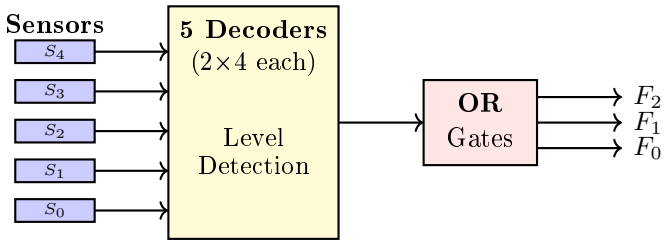


Figure 6: Fuel Sensor System Block Diagram

4.6 Detailed Decoder Connections

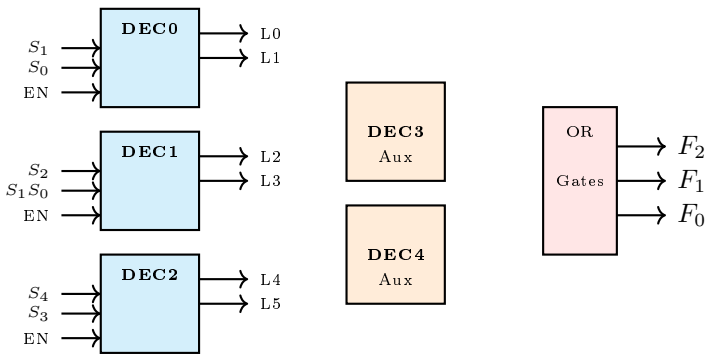


Figure 7: Five 2×4 Decoders Implementation

5 Question 2: Photo Booth FSM (10 marks)

Problem Statement

Design an FSM for a photo booth that accepts nickels (5¢), dimes (10¢), and quarters (25¢). Photo costs **25¢**. Return change if overpaid.

5.1 Input/Output Definition

Inputs	Description	Outputs	Description
N	Nickel (5¢)	P	Photo taken
D	Dime (10¢)	C_5	Return 5¢
Q	Quarter (25¢)	C_{10}	Return 10¢
		C_{15}	Return 15¢
		C_{20}	Return 20¢

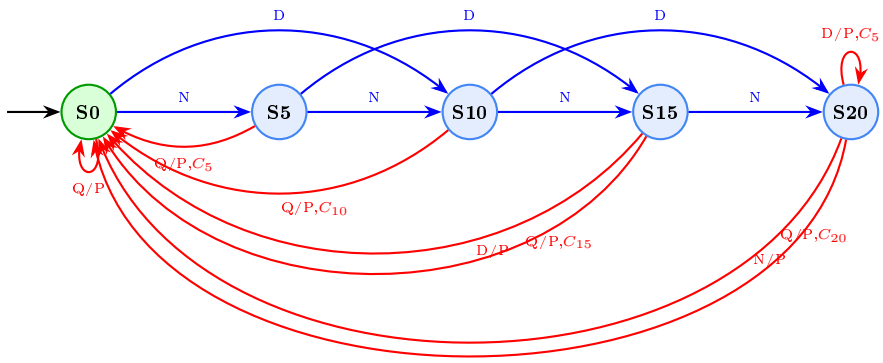
Table 4: Photo Booth I/O

5.2 State Encoding

State	Q_2	Q_1	Q_0	Amount
S0	0	0	0	0¢
S5	0	0	1	5¢
S10	0	1	0	10¢
S15	0	1	1	15¢
S20	1	0	0	20¢

Table 5: State Encoding

5.3 FSM State Diagram



Blue: Add coin Red: Photo taken (return to S0)

Figure 8: Photo Booth FSM (Mealy Machine)

5.4 State Transition Table

State	**Input**	**Next**	P	C_5	C_{10}	C_{15}	C_{20}
S0	N	S5	0	0	0	0	0
S0	D	S10	0	0	0	0	0
S0	Q	S0	1	0	0	0	0
S5	N	S10	0	0	0	0	0
S5	D	S15	0	0	0	0	0
S5	Q	S0	1	1	0	0	0
S10	N	S15	0	0	0	0	0
S10	D	S20	0	0	0	0	0
S10	Q	S0	1	0	1	0	0
S15	N	S20	0	0	0	0	0
S15	D	S0	1	0	0	0	0
S15	Q	S0	1	0	0	1	0
S20	N	S0	1	0	0	0	0
S20	D	S0	1	1	0	0	0
S20	Q	S0	1	0	0	0	1

Table 6: Complete State Transition Table

6 Question 3: Sequence Generator FSM (10 marks)

Problem Statement

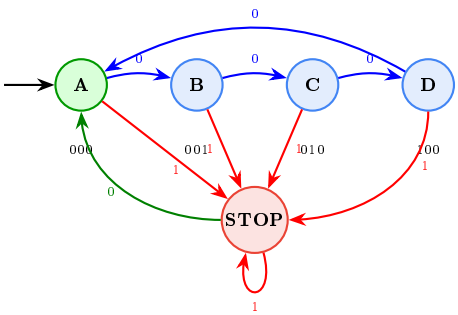
Design FSMs that generate: `000 → 001 → 010 → 100 → (repeat)`
Three variations:

(a) S=1 stops; S=0 restarts from 000

(b) S=1 postpones for 3 cycles, then continues

(c) S=1 postpones for 30 cycles using MSI counter

6.1 Part (a): Stop and Restart



Blue=S:0 (run) Red=S:1 (stop) Green=restart

Figure 9: Part (a): Stop and Restart FSM

6.2 Part (b): 3-Cycle Postpone

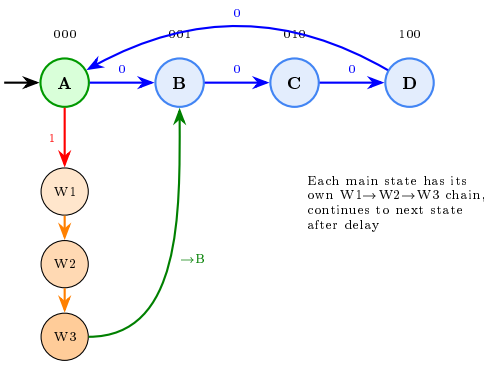


Figure 10: Part (b): 3-Cycle Postpone (showing path from A)

6.3 Part (c): 30-Cycle Delay with MSI Counter

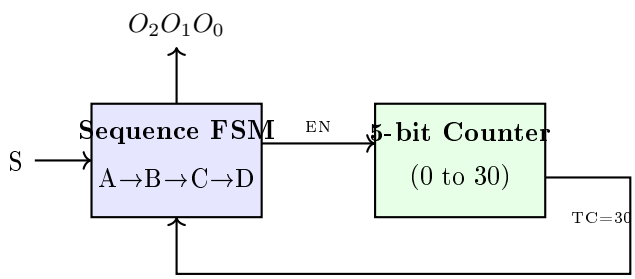


Figure 11: Part (c): FSM with 5-bit MSI Counter

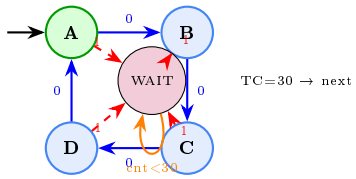


Figure 12: Part (c): FSM State Diagram with Counter

Part III
Quiz Questions

7 Multiple Choice

Q1. A 2×4 decoder with Enable=0 has how many active outputs?

(a) 1 (b) 2 (c) 4 **(d) 0** ✓

Q2. In a Mealy machine, output depends on:

(a) Current state only
(b) **Current state AND input** ✓
(c) Next state
(d) Previous state

Q3. Flip-flops needed for 5 states: $\lceil \log_2(5) \rceil = \mathbf{3}$

Q4. A MUX with 3 select lines can choose from **8** inputs.

Q5. 4-to-16 decoder needs **5** 2×4 decoders (1 for enable + 4 for outputs).

Q6. In sequence 000→001→010→100, which is invalid? **011, 101, 110, 111**

8 Short Answer

SA1. Decoder vs Encoder:

- Decoder: n bits $\rightarrow 2^n$ outputs (one-hot)
- Encoder: 2^n inputs $\rightarrow n$ bits (binary)

SA2. Why use MSI counter for long delays?
Reduces FSM complexity - instead of 30 wait states, use single WAIT state with external counter.

SA3. Moore vs Mealy for photo booth?
Mealy is better - outputs (change) depend on both state (accumulated amount) and input (coin type).

A Quick Reference

A.1 Decoder Expansion

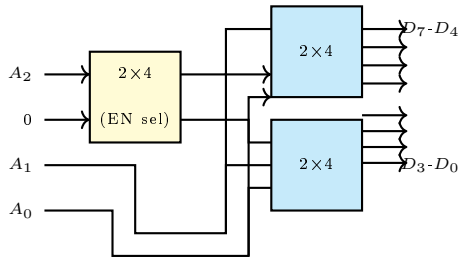


Figure 13: 3-to-8 Decoder from 2×4 Decoders

A.2 FSM Design Checklist

- ☐ Identify inputs/outputs
- ☐ List all states
- ☐ Draw state diagram
- ☐ Choose Moore/Mealy
- ☐ Assign state encoding
- ☐ Create transition table
- ☐ Derive next-state equations
- ☐ Derive output equations
- ☐ Implement circuit

End of Solutions